Fitness Superstore — Inbound Email Triage (Phase 1)

System summary, structure, core rules, and tested routing

---

1. How the system works (high level)

A new email arrives in Gmail. n8n watches the inbox and, for every unread message, walks it through a single workflow that ends in either **a Gmail draft awaiting human approval**, or **a Slack approval request to a named human**. Nothing is ever sent to a customer and no payment is ever approved by AI.

The components, in the order an email touches them:

- **Gmail Trigger** (n8n, OAuth) — polls the test inbox every minute, picks up unread mail.
- **Normalize email** (n8n Code node) — extracts `from / subject / body` into a predictable payload.
- **Google Drive: Search + Download** (n8n) — live-fetches the 12 SOP files from `Fitness Superstore SOPs/Active|Reference|Archived` in Drive. The catalog is loaded on **every run**, so editing an SOP in Drive instantly changes routing without redeploying anything.
- **Build payload** (n8n Code node) — decodes each SOP, tags it Active/Reference/Archived by filename prefix, and bundles the email + 12 SOPs into one JSON request body.
- **OpenClaw HTTP bridge** (server-side Python on `127.0.0.1:8088`) — receives the bundle, writes SOPs to a per-request temp file, and hands off to `triage-chain.py`.
- **The 4-skill OpenClaw chain** (run by `triage-chain.py`) — invokes four independent skills in sequence, validating each output against its JSON Schema before continuing:

  • `classify_email` → picks one of 8 lanes + confidence (validated against `schemas/classify_email.json`)

  • `select_sop` → picks the controlling Active SOP; flags Archived/Active conflicts without auto-escalating (validated against `schemas/select_sop.json`)

  • `draft_response` → writes a customer-safe draft strictly grounded in the matched Active SOP; emits any dodged commitments as `uncommitted_items[]` (validated against `schemas/draft_response.json`)

  • `audit_check` → final independent reviewer; re-checks the no-promise list and forces escalation if any rule is violated (validated against `schemas/audit_check.json`)

  • **If any skill's response fails its schema, the chain stops, populates `schema_errors`, and escalates.**
- **Decision returned to n8n** as the unified routing-decision JSON. The bridge also creates a **Monday card** in the matching lane group with the full reasoning, the `uncommitted_items[]` list, and the schema-error trace (when any) posted as an item update.
- **n8n branches on the decision:**

  • If `action == draft`: n8n's **Gmail Create Draft** node writes the reply to the Drafts folder (never sent).

  • If `action == escalate`: n8n's **Slack Send and Wait for Approval** posts a message to the approver channel with Approve/Reject buttons. The workflow pauses until a human clicks one, then continues based on the response.
- **Surface decision** — a summary item logged in n8n's execution history for audit and debugging.

End state for every email: a Monday card exists (lane + reasoning), and either a Gmail draft awaits a human in the right Drafts folder, or a Slack approval is waiting for the right named approver.

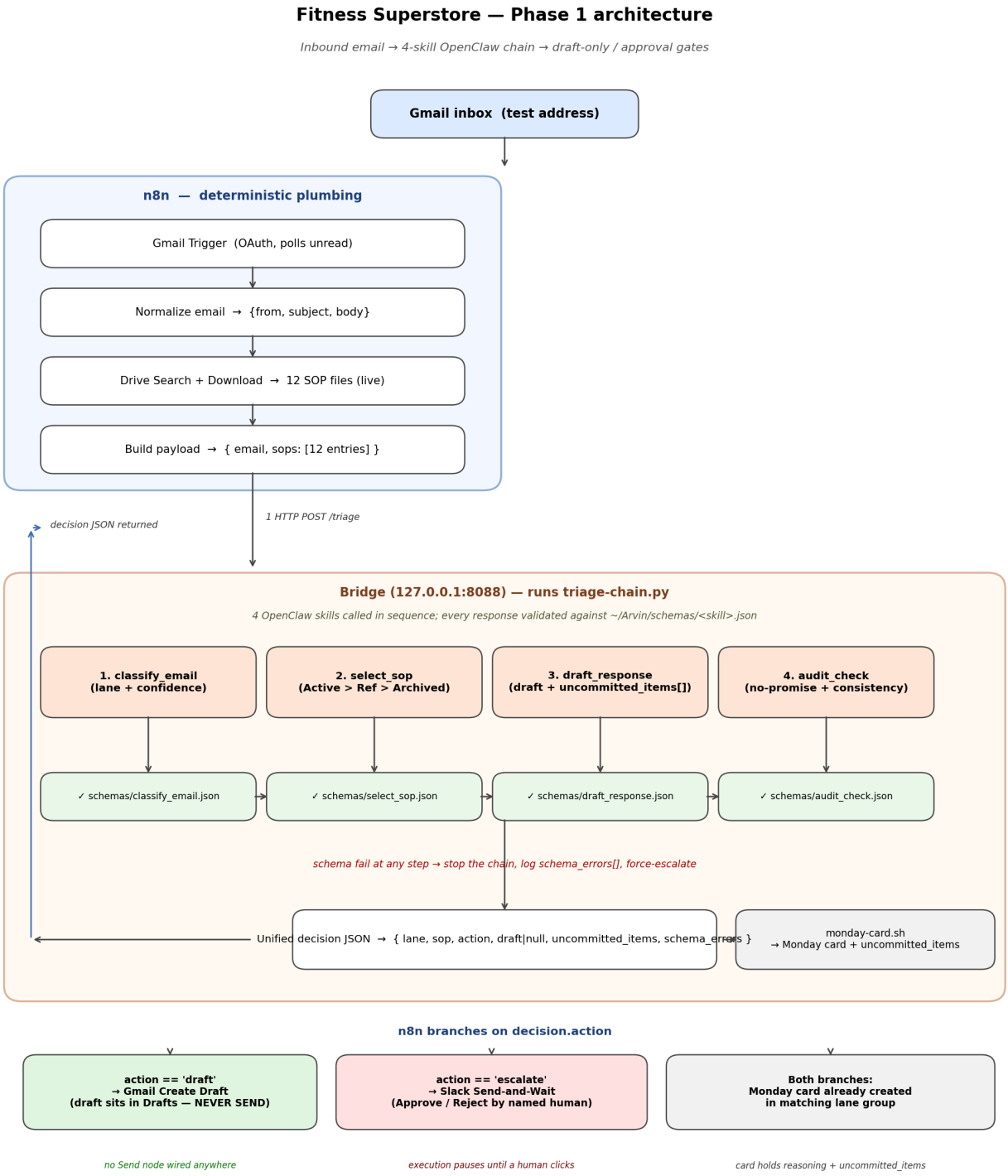

Figure 1 — Phase 1 architecture. Inbound email → n8n plumbing → bridge running the 4-skill OpenClaw chain (with per-skill JSON Schema gates) → Gmail Drafts / Monday card / Slack approval.

---

2. Server-side structure

Everything lives under `/home/yoni/Arvin/` on the Netcup VPS. The repo is organized so each layer is independently inspectable.

```
~/Arvin/
■■■ bin/
■   ■■■ triage-server.py          HTTP bridge on :8088. Receives {email, sops},
```

```
■   ■                          runs the pipeline, returns the decision JSON.
■   ■■■ triage-pipeline.sh      End-to-end orchestrator. Sources env.sh,
■   ■                          calls triage-chain.py, then monday-card.sh.
■   ■■■ triage-chain.py         Runs the 4-skill OpenClaw chain in sequence;
■   ■                          validates each response against its JSON Schema;
■   ■                          handles branching (Finance early-exit, low-
■   ■                          confidence escalate, schema-fail escalate);
■   ■                          returns the unified decision JSON.
■   ■■■ triage-one.sh           Legacy single-orchestrator path. Retained for
■   ■                          manual fixture replay; not used by the bridge.
■   ■■■ extract_decision.py     Lists the routing-decision object out of
■   ■                          OpenClaw's envelope.
■   ■■■ monday-card.sh          Creates the Monday item in the lane's group
■                              and posts the full decision as an item
■                              update, including any uncommitted_items.
■
■■■ config/
■   ■■■ env.sh                  Model + Gmail user + Monday board id.
■   ■■■ monday-groups.json      Lane name → Monday group_id mapping.
■
■■■ schemas/                   Strict JSON Schemas (one per skill);
■   ■■■ classify_email.json     enforced at runtime by triage-chain.py
■   ■■■ select_sop.json         after every OpenClaw call. Schema fail =
■   ■■■ draft_response.json     stop the chain, log the error, escalate.
■   ■■■ audit_check.json
■
■■■ fixtures/                  10 test emails as JSON for offline replay.
■
■■■ n8n/
    ■■■ fitness-triage.workflow.json    Importable workflow.
```

Secrets live outside the repo at `~/secrets/openai.key`, `~/secrets/monday.key`. Tokens are never committed.

---

3. The OpenClaw skills

Judgment is split across **four independent skills**, each living in its own `SKILL.md` under `~/.openclaw/workspace/skills/`. Each skill has a corresponding JSON Schema at `~/Arvin/schemas/<skill>.json` that `triage-chain.py` validates every response against. A schema failure at any step stops the chain and force-escalates.

#	Skill	Job	Strict output (validated against)	
1	`classify_email`	Pick exactly one of 8 primary lanes (+ optional secondary), with confidence and reasoning. Hard-rules ACH → Finance lane @ 100 confidence; low confidence (<70) → Escalate.	`classify_email.json` — enum on the 8 lanes, integer confidence 0-100, no additional properties	

#	Skill	Job	Strict output (validated against)	
2	`select_sop`	Pick the controlling Active SOP. Apply Active > Reference > Archived hierarchy. Flag Archived-vs-Active conflicts in the audit trail but **never escalate solely because an archived SOP exists**.	`select_sop.json` — enforces `use_for_drafting=true` only when `sop_status=Active`, plus the conflict-noted/description invariant	
3	`draft_response`	Write a customer-safe draft strictly grounded in the matched Active SOP. Anything the customer asked for that the draft refuses to commit (refunds, replacements, warranty, delivery dates, fault/blame, technical outcomes) lands in `uncommitted_items[]` for the human approver.	`draft_response.json` — `approval_required` locked to true, approver-role enum, the mandatory `[DRAFT — pending human approval	...]` trailer enforced via regex
4	`audit_check`	Final independent reviewer. Re-checks the no-promise list and consistency across the prior three outputs. Force-escalates on any violation, low confidence, schema mismatch, or Finance lane slipping through.	`audit_check.json` — five-way invariant: `pass=true` ⇔ `force_escalate=false` ⇔ `violations=[]` ⇔ `escalation_reason=null` ⇔ `notify_role=null`	

Why split into four: each skill is independently testable, schema-validated, and re-promptable. A failure in one skill (bad classification, wrong SOP pick, off-tone draft) is caught at the next step rather than cascading into a customer-visible draft.

---

4. Core operating rules (from the brief, enforced everywhere)

These are the rules that make the system safe. Every rule is enforced in at least two of three layers (skill prompt, n8n workflow, server scripts), so violating any single one would take a multi-layer bypass.

Rule	Where it's enforced
Only Active SOPs control routing	Skill prompt + decision schema (`controlling_sop.status` is always reported)
Reference SOPs are context only	Skill prompt; Reference SOPs never appear as `controlling_sop`
Archived SOPs must not control routing	`select_sop` SKILL.md + the rule that Active wins, archived flagged in `sop_conflict.detected` for audit (Stage 2: do **not** escalate solely because an archived SOP exists when an Active SOP clearly controls)
AI may classify, summarize, recommend, draft — but no more	Strict output schema; nothing outside it is consumed by the workflow
AI must NOT auto-send customer emails	Structural: n8n workflow has no Send / SMTP / Reply node, only Gmail "Create Draft"
ACH / payment requests require Owner approval	Hardcoded skill rule + decision JSON forces `action: escalate` + `approver_role: Owner`
One email = one primary workflow	Skill prompt + decision schema (single `primary_lane`)
Escalate if source is conflicting, missing, or unclear	`classify_email` hard rule: confidence < 70 forces lane = Escalate; `select_sop` escalates when no usable Active SOP exists; `audit_check` force-escalates on any no-promise violation, schema mismatch, or upstream inconsistency
Draft must never promise refund amounts, replacements, warranty coverage, specific delivery dates, fault/blame, or technical outcomes	`draft_response` no-promise list + `uncommitted_items[]` for the human; `audit_check` re-verifies the body and force-escalates on violation
Every OpenClaw response must match a strict JSON Schema	`triage-chain.py` validates each skill output against `~/Arvin/schemas/<skill>.json` after every call; mismatch → stop the chain, populate `schema_errors`, escalate

---

5. The 8 workflow lanes

- **New Orders / Pending Shipments** — paid but unfulfilled, ETA questions
- **Shipping CS** — freight (LTL) tracking, delivery scheduling post-pickup
- **Parcel / Small Package** — UPS / FedEx / USPS small-package issues
- **Install / Service / Warranty** — installation, post-delivery damage, warranty
- **Sales CS** — pre-sale questions, financing, availability
- **Finance / ACH / Owner Approval** — vendor ACH, payment confirmation (always escalate)
- **Product Content / Ecommerce** — internal reports of bad product pages
- **Escalate / Needs Human Review** — anything unclear, conflicting, low-confidence

---

6. Routing logic — the hard discriminators

The classifier is not freeform. Each of the 8 lanes is defined by a specific keyword/intent discriminator so the brain can never collapse two distinct workflows into one. The ones that matter:

• **Freight (LTL) vs Parcel (small package)** — different carriers, different lanes, different claim processes. Freight clues: *LTL, PRO number, freight, liftgate, pallet, scheduled delivery, large equipment*. Parcel clues: *UPS, FedEx, USPS, tracking 1Z…, doorstep delivery*. These never merge. A "where's my stuff" email is **Shipping CS** if freight, **Parcel** if parcel — the carrier identity is the discriminator, not the question.

• **New Orders vs Shipping CS** — paid but no tracking yet → New Orders. Paid + tracking exists + asking about delivery → Shipping CS. The presence/absence of a PRO/tracking number is the signal.

• **Install + delivery in the same email** → primary lane is **Install / Service / Warranty**, with Shipping CS noted secondary. Never split into two workflows.

• **Sales CS (financing) vs Finance/ACH/Owner** — consumer pre-sale financing is **Sales CS**; vendor ACH / payment / bank-detail change is **Finance / ACH / Owner Approval**. Mixing these would route a real fraud vector to the wrong human.

• **Active vs Reference vs Archived SOP** — only Active controls. Reference adds tone/context. Archived must be IGNORED for routing AND flagged as a conflict in the decision JSON for the audit trail. **The presence of an archived SOP alone is not enough to escalate** — if a clear Active SOP exists, the Active SOP is used and the conflict is just recorded.

• **Out-of-lane / non-customer noise** (LinkedIn, marketing, account alerts) → always escalate. Low classifier confidence also forces escalate even when one lane looks close.

---

7. Safety reasoning — four-layer enforcement

Each safety rule is enforced in multiple independent layers so no single bypass can violate it. The principle is **defense in depth**: a prompt bug, a workflow misconfig, an off-spec AI response, or a single bad commit shouldn't be enough to break the safety contract.

• **Layer 1 — the four OpenClaw skill prompts.** Hard rules embedded in each `SKILL.md`: ACH always escalates, refunds never commit a dollar amount, archived SOPs never control routing, every draft must end with the mandatory `[DRAFT — pending human approval | ...]` trailer. Each skill is independently re-promptable without touching the others.

• **Layer 2 — JSON Schema validation in the chain.** After every OpenClaw call, `triage-chain.py` validates the response against `schemas/<skill>.json` using the `jsonschema` Python library. A schema violation never makes it past the orchestrator: the chain halts immediately, the offending error is recorded in `schema_errors[]`, and the workflow is forced into the escalate path with a Schema-validation reason.

• **Layer 3 — the `audit_check` final reviewer.** Even when all earlier schemas pass, the fourth skill independently re-checks the draft body against the explicit no-promise list (refund amounts, replacement promises, warranty confirmations, specific delivery dates, fault/blame statements, technical outcome promises). Any violation force-escalates with a named `notify_role`.

• **Layer 4 — n8n workflow structure.** The workflow only contains a Gmail `Create Draft` node — there is no `Send`, no SMTP, no `Reply` action wired anywhere in any branch (and not even one left as disabled, per Stage 2). ACH and other escalations go through `Send and Wait for Approval` which structurally requires a human button-click before the next node runs.

---

8. Approval matrix — who approves what

The matrix mirrors real operational risk: the higher the blast radius of a wrong action, the higher the named approver.

Action AI proposed	Who approves before action	How the gate is enforced
Classification, summary, routing decision	none (AI autonomous)	n/a
Draft customer reply (low-risk lane)	CS agent	Draft sits in Drafts; no send node exists
Refund / damage reply (Case 4)	CS Lead	`approval_required: true`, `approver_role: CS Lead`; draft never names an amount
Sending any customer email	CS agent / CS Lead	Architectural: only Create Draft is wired in the workflow
Vendor ACH / payment / bank change	**Owner** (out-of-band phone verification mandatory)	Hardcoded escalate + holding draft commits to nothing
Archived SOP detected against active routing	surfaced in the decision JSON + Monday update	`sop_conflict.detected: true` with the archived SOP id named in the audit trail (no escalation if Active SOP clearly controls)
Customer asked for something the draft refuses to commit	recorded in `uncommitted_items[]` and shown to the human approver	The Monday card update lists each dodged ask; the Slack approval message includes the same list before the Approve/Reject buttons

Principle: AI proposes, a named human disposes. The riskier the action, the higher the approver. ACH and refunds — the two paths that move money — both have a named human in the loop *before* anything customer-visible happens.

---

9. Workflow thinking — why this architecture

Some choices that aren't obvious from looking at the workflow:

• **Judgment in four separate OpenClaw skills, plumbing in n8n.** Each step requiring reasoning (classify, pick SOP, write draft, audit-check) lives in its own skill with its own schema, so a failure or rewrite of one doesn't drag the others. Every deterministic step (poll Gmail, fetch Drive, create Monday card, post to Slack) lives in n8n. This keeps the AI surface area small and auditable, and lets prompts evolve without re-deploying the workflow.

• **Schema validation lives in the bridge, not n8n.** Stage 2 requires every OpenClaw response to be schema-validated before the next step runs. We put the validation in `triage-chain.py` (using Python's `jsonschema` library) rather than spreading 4-8 Code nodes across n8n. The n8n workflow stays simple; the bridge returns either a valid decision or a populated `schema_errors[]` and force-escalates. Same outcome, far fewer moving parts.

• **SOPs live in Drive, not in code.** Live-fetched on every workflow run. Editing SOP-04 in Drive instantly changes how the next damage email is drafted — no code change, no redeploy. The Archived folder is a real subfolder, not a feature flag, so its state can be inspected by ops outside the engineering pipeline.

• **Monday card is the audit log; Slack is the approval surface.** Monday is slow-acting and persistent — it's the queue, the audit trail, the report-from view. Slack is fast-acting and ephemeral — it's where the

approve/reject click happens. Both are required for different reasons; replacing one with the other loses something important.

• **Approval is synchronous (Send-and-Wait), not async polling.** When an ACH escalation fires, the n8n execution literally pauses on the Slack node until a human clicks. That pause is recorded in the Executions panel — there's no "did anyone approve this?" question, the workflow either resumed or it didn't. Async polling can lose state; pausing the execution can't.

• **No Send node anywhere.** This is the single most important architectural choice. The system is **structurally** incapable of sending a customer email, so even a worst-case AI mistake — a hallucinated "ok, sending the refund now" decision — does literally nothing customer-visible. Safety becomes a property of the wiring diagram, not of the AI's behavior on any given run.

---

10. Tested cases summary

All 4 intended brief cases were exercised end-to-end (live email → n8n → OpenClaw → Drafts/Slack/Monday):

• Customer ETA (case 1) → drafted, SOP-01, Monday card in New Orders group, `uncommitted_items` populated with the dodged ship-date ask

• Vendor ACH (case 6) → escalated, Slack approval, Owner role, holding draft on approve

• Archived-SOP freight conflict (case 10) → SOP-02 (Active) applied, ARCH-01 named in `sop_conflict.detected` audit trail, draft created per Stage 2 archive-conflict rule (no escalation)

• Damage + refund demand (case 4) → drafted empathetically with CS Lead approval gate, no dollar amount committed

Each of these was re-run through the new 4-skill chain (`triage-chain.py`) with full per-skill schema validation; every response passed its `schemas/<skill>.json` check and `schema_errors[]` came back empty.

---

11. Permission matrix

Every external system the workflow touches, with the **minimum** access required for Phase 1. Tokens are stored under `~/secrets/` (file mode 600) and never committed to the repo. n8n credentials are scoped to the same accounts.

System	Read scope	Write scope	Credential / scope	Approver	Phase
Gmail	INBOX (filtered to unread)	Drafts folder only — never `Send` / `Reply`	OAuth2 — `gmail.readonly` for trigger + `gmail.compose` for Create Draft	n8n owner	1
Google Drive	`Fitness Superstore SOPs/{Active,Reference,Archived}/*.md`	none	OAuth2 — `drive.readonly`, scoped to the single SOP folder	n8n owner	1

System	Read scope	Write scope	Credential / scope	Approver	Phase
Monday.com	items + columns on the CS Triage board	items, item updates, status-column changes on the CS Triage board	API token, board-scoped	Board owner	1
Slack	the approver channel only	post message + buttons in the approver channel	Bot token — `chat:write`, `chat:write.public`, `channels:read`, `users:read`	Workspace admin	1
OpenAI (via Open Claw)	n/a	n/a — outbound model call only	API key at `~/secrets/openai.key`, mode 600	dev	1

Notes:

- **No `Send` / `Reply` / SMTP scope anywhere.** Gmail's `gmail.compose` permits drafting but not sending; the n8n workflow never wires a Send node.
- **No payment, refund, or money-moving scope anywhere.** Nothing in the credential set can move money — ACH escalations exist purely to put the question in front of a named Owner for out-of-band action.
- **Drive is read-only** and scoped to the SOP folder. The system cannot write, delete, or rename SOP files; SOP edits stay an explicit human action in Drive's UI.
- **Slack is scoped to a single approver channel** and to the bot's own messages. The bot cannot read message history outside the channel or DM anyone.

---

12. Logging / audit plan

Every step that matters for compliance, debugging, or "who approved what" leaves a trace. The principle is **three-tier durability** — fast-rotating diagnostic logs, persistent per-request artifacts, and human-readable audit cards.

Tier 1 — diagnostic stderr (per-request, ephemeral):

- `triage-chain.py` prints `[chain] <skill> -> <output>` to stderr after every skill call.
- The HTTP bridge captures stderr of each pipeline run into `~/Arvin/out/<request-id>.triage.err`.
- Useful for debugging a single request; rotated/cleaned weekly.

Tier 2 — per-request decision JSON (persistent, structured):

- The complete decision JSON for every request is written to `~/Arvin/out/<request-id>.decision.json`.

• The corresponding inbound `email.json` and Drive-fed `sops.json` live at `/tmp/<request-id>.{json,sops.json}`.

• These together let any auditor reconstruct exactly what the AI saw, what each skill returned, whether each schema passed, and what decision was emitted.

• Retention: ≥ 90 days; archived to cold storage after that for the compliance window the company requires.

Tier 3 — Monday card update (human-readable, permanent):

• Every triage emits one Monday item in the lane group with the full reasoning, controlling SOP id + status, confidence, approver role, conflict flag, **and the** `uncommitted_items[]` **list** posted as the item update body.

• The Monday item retains its full timeline: who changed the status column, who commented, when the draft was created or sent.

• This is the audit trail the company would show in a quarterly review.

Special-case logging:

• **Schema validation failures** — populate `schema_errors[]` in the decision JSON, set `action: escalate`, and the failure is surfaced both in the Monday card body and in the Slack approval message. Easy to query: "how many of last week's emails had schema failures, and which skill?"

• **Slack approval timing** — recorded automatically by n8n's Executions panel: timestamp the message was posted, timestamp Approve/Reject was clicked, who clicked it. Available retrospectively for any specific decision.

• **No customer-PII redaction needed in Phase 1** — sandbox uses synthetic emails. Production design would add a redaction step before any logs leave the box.

---

13. Phase 1 build plan

Goal: deliver the Phase 1 sandbox demo end-to-end on a single Netcup VPS, with a clear path to a controlled production cutover. Estimated effort below assumes one developer + occasional architecture review.

Milestones

Milestone	What lands	Effort	Status
M0 — Infra ready	n8n + OpenClaw on Netcup VPS, secrets at `~/secrets/`, UFW deny-by-default, SSH-tunnel access	—	■ done
M1 — Single-skill triage	`fitness-triage` orchestrator skill + bridge + monday-card.sh + workflow JSON; case01/04/06/10 verified end-to-end	—	■ done in Stage 1

Milestone	What lands	Effort	Status
M2 — Stage 2 split	4 separated skills + 4 JSON Schemas + `triage-chain.py` with per-step validation; `uncommitted_items[]` flowing into Monday and Slack	—	■ done
M3 — Documentation packet	Permission matrix, audit plan, this build plan, 10-case test result table, fresh n8n workflow export, updated architecture diagram	~1 day	■ in progress
M4 — Production access plan	Stakeholder approval for real Gmail + Drive scopes; OAuth verification, named approver identities for CS Lead / Ops Manager / Owner; Monday board provisioned in real workspace	~3 days (mostly waiting on access reviews)	■ pending
M5 — Controlled go-live	Real Gmail address pointed at production n8n; first ~20 emails reviewed manually by CS Lead before relaxing oversight; Slack channel set to real approver group	~2 days	■ pending
M6 — Post-launch tuning	Confidence calibration, SOP prompt tuning, no-promise rule refinement based on real escalation patterns	ongoing	■ pending

Required access for production cutover (M4):

- Real customer-facing Gmail inbox + OAuth grant from the inbox owner
- Drive folder structure provisioned in the real workspace with the same Active / Reference / Archived hierarchy
- Real Monday board in the company's workspace with the 8 lane groups + Approval status column
- Slack workspace + dedicated approver channel; bot installed and invited
- OpenAI / Anthropic API key on the company's billing account (move off the demo key)

Risks (and mitigations):

Risk	Likelihood	Mitigation
Real customer emails contain PII the sandbox never saw	High	Add a redaction step before the OpenClaw call; restrict log retention; verify with Legal before go-live
Active SOP changes don't immediately propagate	Low	Already live-fetched from Drive on every run — no redeploy needed; verified
AI starts hallucinating off-skill behavior under unfamiliar input	Medium	Schema validation catches structural drift; `audit_check` catches no-promise violations; live errors visible in `schema_errors[]` and surface in Slack
Slack approval bottleneck — approver out of office, escalations pile up	Medium	Add a secondary approver fallback (notify second person after N minutes); track Slack approval latency as a KPI
Cost overrun on AI calls (4 per email × volume)	Low	Move from `gpt-5.5` to a cheaper classifier for `classify_email` if confidence stays high; current per-email cost is acceptable for sandbox
Schema-prompt drift after model upgrade	Medium	All schemas have `additionalProperties: false`; any new model behavior that adds fields is immediately rejected with a clear error

Phase 2 candidates (intentionally out of Phase 1 scope, documented for the roadmap):

- Shopify integration — read-only order + fulfillment lookup before classify_email
- Gorgias integration — pull recent ticket history into the email payload for channel-conflict detection
- Auto-redaction of customer PII in logs
- Slack interactive buttons (vs reactions) once n8n has a public HTTPS endpoint
- Multilingual draft support
- Monday automation: auto-close cards 24h after the draft is sent

---

14. 10-case test results

All 10 brief fixtures were re-run through the new 4-skill chain (`triage-chain.py`) end-to-end. Each row is the actual decision JSON the chain emitted for that case (raw files: `/tmp/stage2_runs/case0N.decision.json` on the server). Every response across all 40 individual skill calls passed its JSON Schema — total `schema_errors` across all 10 runs: **0**.

#	Case (short)	Lane returned	Controlling SOP (status)	Conflict	Action	Approver	Conf.	Uncommitted	Schema errs	Stage 2 ✓
1	Unshipped order ETA	New Orders / Pending Shipments	SOP-01 (Active)	no	draft	CS Lead	92	1	0	✓
2	Freight tracking after pickup	Shipping CS	SOP-02 (Active)	yes → ARCH-01 (followed Active)	draft	CS Lead	98	1	0	✓
3	Mixed shipping + install	Install / Service / Warranty	SOP-03 (Active)	no	draft	CS Lead	96	2	0	✓
4	Damage + refund	Install / Service / Warranty	SOP-04 (Active)	no	draft	CS Lead	95	1	0	✓
5	Pre-purchase financing	Sales CS	SOP-05 (Active)	no	draft	CS Lead	98	1	0	✓
6	Vendor ACH request	Finance / ACH / Owner Approval	SOP-06 (Active)	no	**escalate**	**Owner**	100	—	0	✓
7	Product page issue (internal)	Product Content / Ecommerce	REF-01 (Reference)	no	escalate	CS Lead	100	—	0	✓
8	Gorgias vs Gmail conflict	Escalate / Needs Human Review	SOP-08 (Active)	no	escalate	CS Lead	98	—	0	✓
9	Parcel "delivered" but missing	Parcel / Small Package	SOP-07 (Active)	no	draft	CS Lead	98	2	0	✓
10	Freight tracking vs archived SOP	Shipping CS	SOP-02 (Active)	**yes → ARCH-01 (followed Active, drafted per Stage 2 rule)**	**draft**	CS Lead	98	0	0	✓

Key safety observations from the runs:

- **Auto-fail #1 (AI approves ACH)** — Case 6 hit at confidence 100, escalated to Owner, no draft generated by `draft_response`. Holding draft only emerges after a human Approves in Slack.
- **Auto-fail #2 (AI auto-sends)** — Structurally impossible regardless of any decision; n8n has only `Gmail → Draft → Create` wired in either branch.
- **Auto-fail #3 (archived controls routing)** — Case 10 detected ARCH-01, followed Active SOP-02, and produced a draft (not an escalation), exactly as Stage 2 §0 requires. Case 2 also independently flagged ARCH-01 as a defensive note even though the email didn't explicitly trip the archive trap.
- **Auto-fail #4 (freight vs parcel confusion)** — Case 2 → Shipping CS (freight/LTL), Case 9 → Parcel / Small Package (UPS). Never crossed.
- **Auto-fail #5 (generic AI-flavored answer)** — Each row above cites a specific SOP id + status; classify_email also returned grounded `signals[]` extracted from the email body (omitted from the table for brevity).

Reproduction: `bash /tmp/run_all_10.sh` on the server (or manually `python3 /home/yoni/Arvin/bin/triage-chain.py <fixture> /tmp/test-sops.json` for any one case). Each run produces a per-case decision JSON and a stderr trace showing the four skill outputs in sequence.

In addition, three unsolicited "noise" emails arrived during testing — Google security alert, a monday.com onboarding email, and a LinkedIn job-alert digest — and all three were correctly classified as out-of-scope and escalated (SOP-00 Master Policy). This is exactly the safety behavior the spec asks for: when nothing matches, escalate rather than draft.

The full Slack approval loop was verified end-to-end: ACH email arrives → Slack message posted to `#n8n-alerts` via the `n8n-alert` bot → human clicks Approve → Gmail draft created in the Drafts folder of the test inbox → nothing ever appears in Sent.